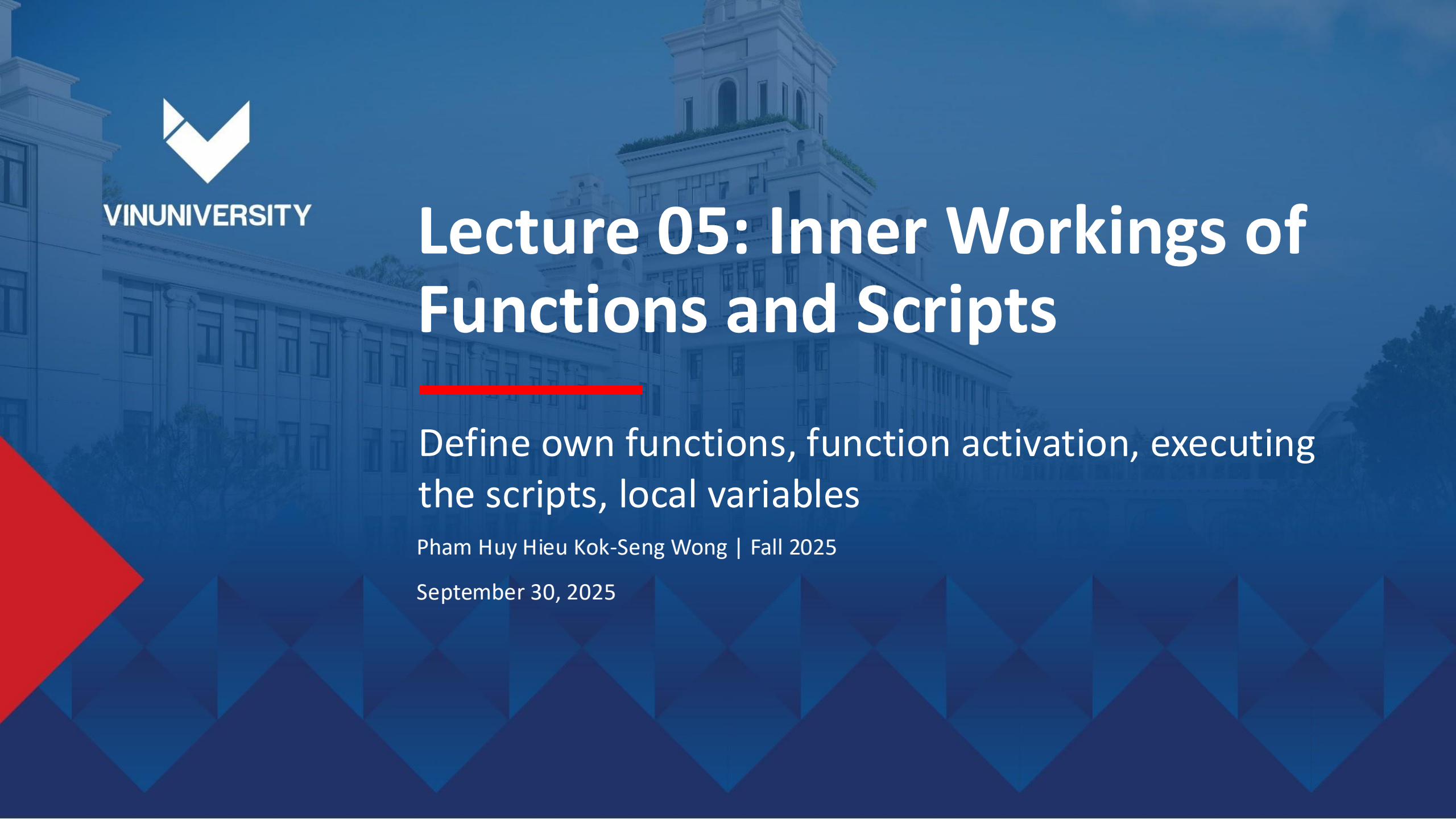


VINUNIVERSITY

Lecture 05: Inner Workings of Functions and Scripts

Define own functions, function activation, executing the scripts, local variables

Pham Huy Hieu Kok-Seng Wong | Fall 2025

September 30, 2025

COURSE ANNOUNCEMENT

Create a 20–30 minute instructional video in which your group demonstrates and explains a Python programming concept covered in Weeks 1–6



VINUNIVERSITY

College of Engineering and Computer Science
COMP1010 Introduction to Programming

COMP1010: Introduction to Programming

Video Assignment 1 Instructions

Introduction

- Groups will be assigned by the instructor, with 4–5 students in each team.
- Release Date: September 27, 2025 (Week 03).
- Due Date: October 27, 2025 (Week 07).
- Weight: 5% (out of 10% total for video assignments).
- Submission Format - The group leaders must upload their group's video file (before the due date) to:
  1. Canvas (assignment submission page) and
  2. Facebook Group (COMP1010 - Fall 2025) - please create a post with topics and team member information (e.g., tag all group members).

Late submissions will be penalized in accordance with the course policy.

LECTURE 04 (RECAP)

In lecture 04, students have:

- understood function characteristics and function calls in Python
- used the standard library in Python
- written and used built-in functions, and
- been introduced to external libraries in Python.

MOTIVATIONAL EXAMPLE 1

Input → Process → Output



[source]



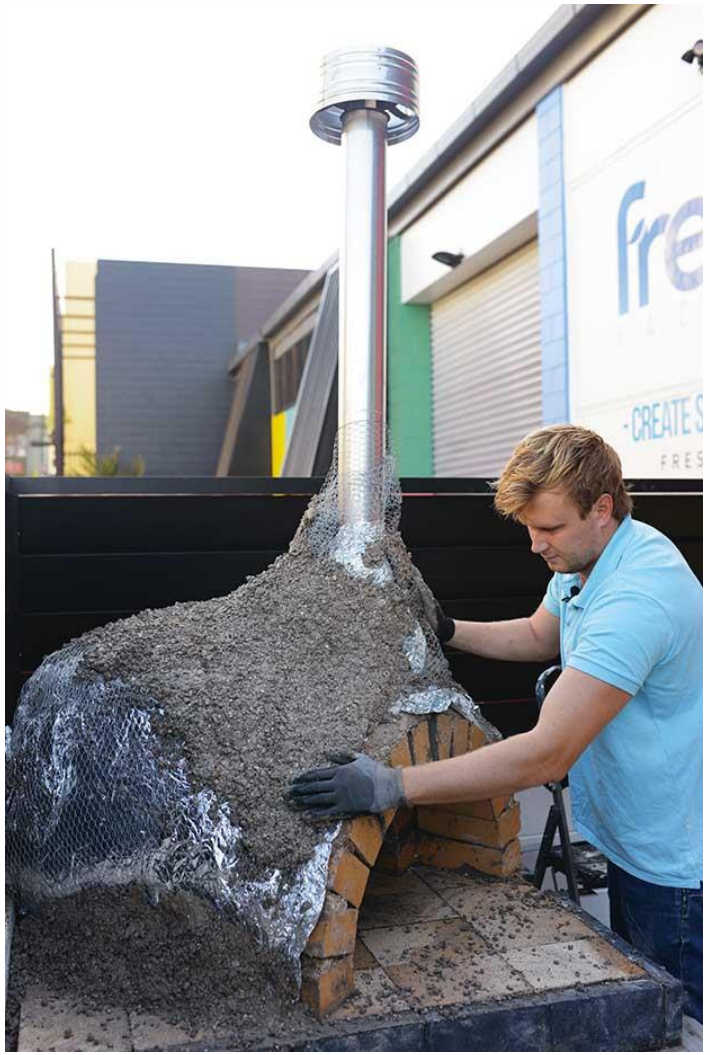
[source]

MOTIVATIONAL EXAMPLE 2

Building your own pizza oven



[source]



[source]

LEARNING OUTCOMES

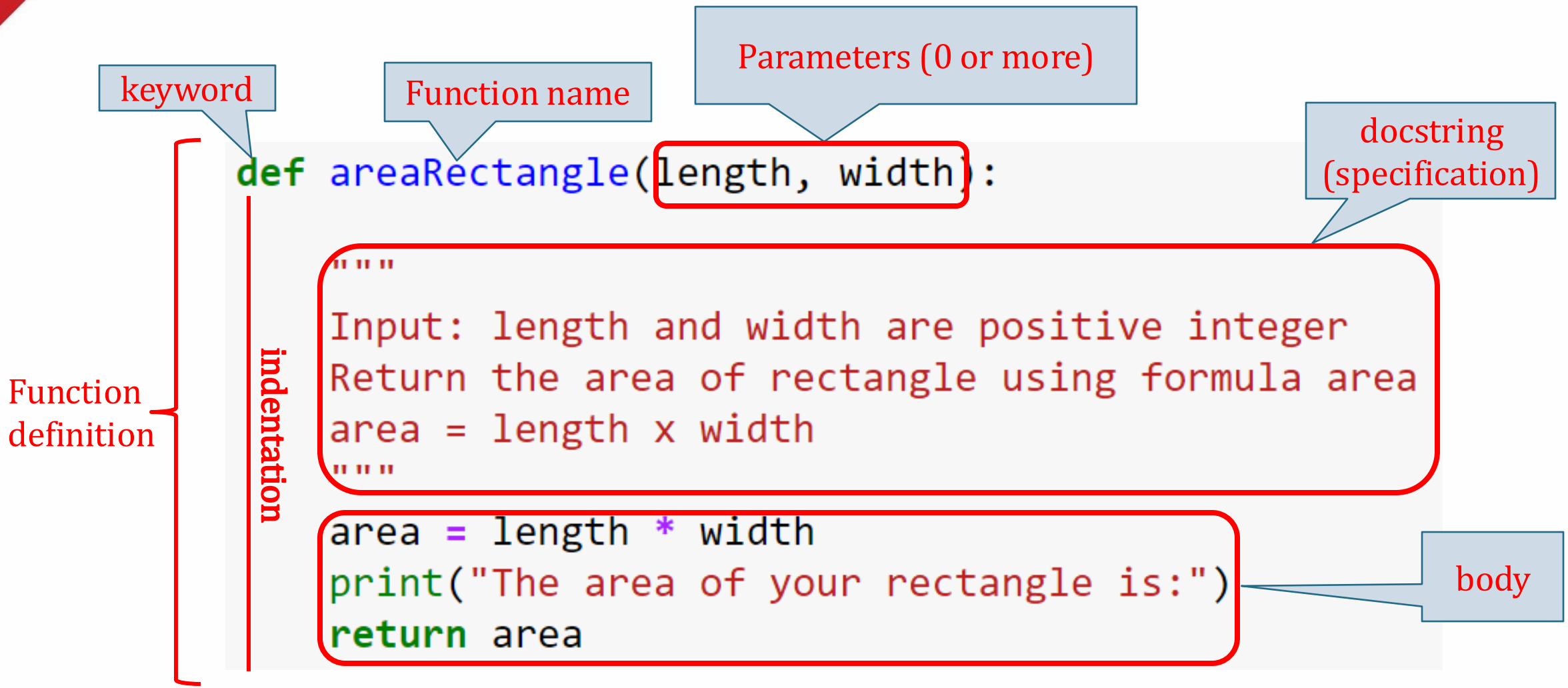
Upon the completion of this lecture, students will be able to:

- understand how to create own function and explain the anatomy of a function definition.
- describe the order in which statements are executed, i.e., flow of execution.
- use Python Tutor to understand your programs by visualizing how code runs step-by-step.

ADDING NEW FUNCTIONS

- Once a function is defined, it can be reused over and over throughout our program.
- The rules for function names are the same as for variable names. E.g., cannot use a **keyword** as the name of a function.
- Avoid having a variable and a function with the same name.

ANATOMY OF A FUNCTION DEFINITION



FUNCTION HEADER AND BODY

Function header

```
def areaRectangle(length, width):

    """
    Input: length and width are positive integer
    Return the area of rectangle using formul
    area = length x width
    """
    area = length * width
    print("The area of your rectangle is:")
    return area
```

Run some commands

keyword

Expression to evaluate and return

```
>>> type(areaRectangle)
<class 'function'>
```

THE RETURN STATEMENT (RECAP)

- Format: `return<expression>`
- Any statement after `return` are ignored
- Passes a value from the function to the caller

```
def areaRectangle(length, width):
    area = length * width
    print("The area of your rectangle is:")
    return area
```

- If absent, special value None will be returned

```
def areaRectangle(length, width):
    area = length * width
```

OR

```
def areaRectangle(length, width):
    area = length * width
    return
```

FUNCTION DEFINITIONS VS FUNCTION CALLS

Function definition

- Define what function does
- Declaration of parameter `base` and `height`

```
def areaTriangle(base, height):

    """
    Input: base and area are positive integer
    Return the area of a triangle using formula area
    area = 0.5 x base x height
    """
    print("The area of your triangle is:")
    return 0.5 * base * height

output = areaTriangle(5, 10)
print(output)
```

Storing returned value

Function call

- Command to do the function
- Argument to assign to `base` and `height`
- Argument: a value to assign to the function parameter when it is called

USER-DEFINED FUNCTION EXAMPLE 1

python

```python
def add_numbers(x, y):
    """
    This function adds two numbers and returns the result.

    Parameters:
        x (int): The first number.
        y (int): The second number.

    Returns:
        int: The sum of x and y.
    """
    result = x + y
    return result

# Calling the function and storing the result
sum_result = add_numbers(5, 3)

print(sum_result)  # Output: 8
```

USER-DEFINED FUNCTION EXAMPLE 2

In what situation do we need no parameter for a function?



USER-DEFINED FUNCTION EXAMPLE 2

In what situation do we need no parameter for a function?

```
def greet():
    print("Hello, VinUnian!")

    # Calling the function
    greet()
```



USER-DEFINED FUNCTION EXAMPLE 2

```
def greet(name):
    print("Hello, " + name)


greet("Alice")


result = greet("Bob")
print(result)
```

OUTPUT?

```
Hello, Alice
Hello, Bob
None
```



USER-DEFINED FUNCTION EXAMPLE 3

```
def add(a, b):
    return a + b



add(5, 3)
```

OUTPUT?

The function add(5, 3) will execute and return 8, but the return value is discarded since there is no variable to store the result.



FLOW OF EXECUTION

Flow-of-execution: The order in which statements are executed.

- Execution always begins at the first statement of the program.
- Statements are executed one at a time, from top to bottom.
- Function definitions do not change the flow of execution of the program - statements inside the function are only executed once the function is called.

PYTHON FUNCTIONS VS PYTHON METHODS

- Python function is part of the Python script file, whereas methods are defined inside a class definition.
- We can call a function directly if it's in the same module or import a module and then call the function directly.
- To call a method, we need a class or an object of the class.
- Python functions can access all the global variables, whereas Python class methods can access global variables, class attributes, and functions.

METHODS VS. FUNCTIONS

"In Python, both functions and methods are blocks of reusable code designed to perform specific tasks."



What's the difference between a method and a function?

METHODS VS. FUNCTIONS

- Functions: are independent and called by their name.
- Methods: are associated with classes/objects, called on an object of the class, and typically operate on that object's data.

```
class Message:
    def hello(self):
        print("I am a method!")
```

```
def hello():
    print("I am a function!")
```

```
# calling a function
hello()

# calling a method
m = Message()
m.hello()

# checking data types
print(type(hello))
print(type(m.hello))
```

```
I am a function!
I am a method!
<class 'function'>
<class 'method'>
```

DEFAULT VALUE FOR FUNCTION PARAMETER

```
def lab(group = "G1"):
    print(f'Your lab session is {group}.')
```

DEFAULT VALUE FOR FUNCTION EXAMPLE

```
def add(a, b=5):
    return a + b

print(add(3))
```

```
def add(a=5, b):
    return a + b

print(add(3))
```

SyntaxError: non-default argument follows default argument

PROBLEM/ISSUE

```
def add(a=5, b):
    return a + b

print(add(3))
```

- In the function `add(a=5, b)`, `a` has a default value, but `b` does not.
- Python expects any non-default argument to appear **before** any default argument.
- Since `b` has no default value, it must come before `a=5`, which has a default value.



CORRECTED CODE

You need to switch the order of the arguments, or provide a default value for `b` as well.

Option 1: Move the non-default argument before the default argument

python Copy code

```python
def add(a, b=5):
    return a + b

print(add(3))  # Output: 8
```

Option 2: Provide a default value for both arguments

python Copy code

```python
def add(a=5, b=0):
    return a + b

print(add(3))  # Output: 3
```

In the corrected version, the function will now work because either both arguments have default values, or the default value comes **after** non-default arguments.



CLASS PRACTICE

Sample Code:

```
def mean(t = 200, n = 10):
    return t / n


x = mean()
y = mean(100)
z = mean(100, 2)
k = mean(n = 5)


print(x, y, z, k)
```

Output:



CALLING FUNC FROM ANOTHER FUNC

Example 1:

```
def sum(a, b):
    return a + b

def avg1(x, n):
    return x / n

y = sum(12, 14)
avg1(y, 2)
```

Example 2:

```
def sum(a, b):
    return a + b

def avg2(c, d, n):
    y = sum(c, d)
    return y / n

avg2(12, 14, 2)
```

NESTED FUNCTION

- Python supports nested functions (also known as inner functions).
- In other words, defining a function inside another function.

```
def func1():
    print ("Hello, I am from the outer function")

    def func2():
        print ("Hello, I am from the inner function")
    func2()

func1()
```

Outer Function

Inner Function

```
Hello, I am from the outer function
Hello, I am from the inner function
```

NESTED FUNCTION EXAMPLE

- Nested Function: A function defined inside another function

```
def say_hi(first, last):
    def getFullName():
        return first + " " + last

    print("Hi, " + getFullName() + "!")

say_hi("Ali", "Baba")
```

CLASS PRACTICE

Solve the following problems by defining a function.

Problem 1

Write a program that asks for the user's year of birth and print out the age in the following format:

```
Your age is 25
```

Problem 2

Write a program to ask for the number of students in a class. Next, request the user to enter the total score of all students and then output the average exam score of the class.

EXECUTING THE SCRIPT - EXAMPLE

Python skips

Python skips

Python learns about the function

Python skips everything inside the function

Python executes the function body

Python executes this statement

```
#areaTriangle.py

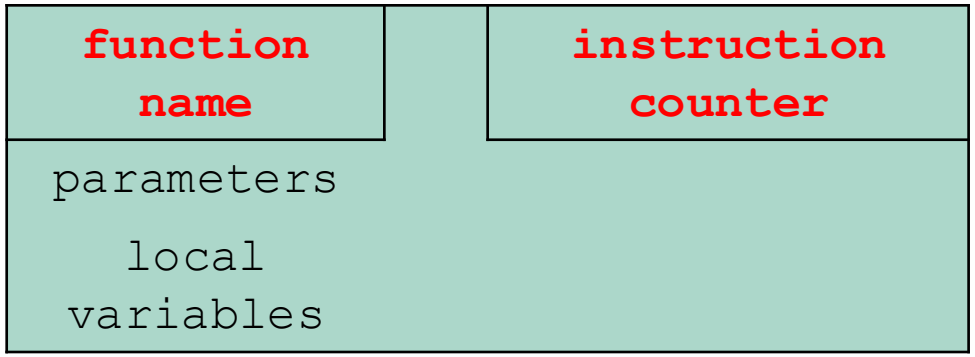
"""
script that compute the area of a triangle
"""

def areaTriangle(base, height):
    """
    Input: base and area are positive integer
    Return the area of a triangle using formula area
    area = 0.5 x base x height
    """
    print("The area of your triangle is:")
    return 0.5 * base * height

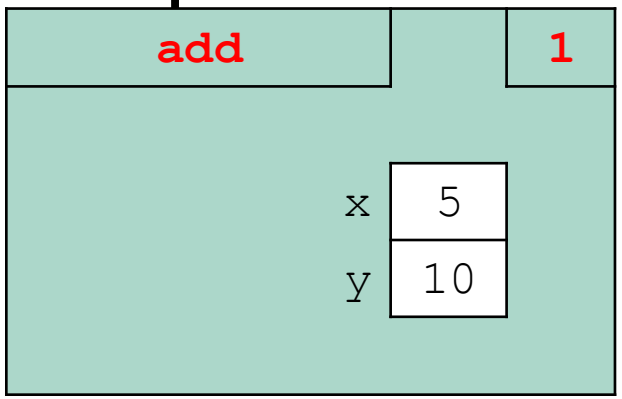
areaTriangle(5, 10)
```

UNDERSTAND HOW FUNCTIONS WORK

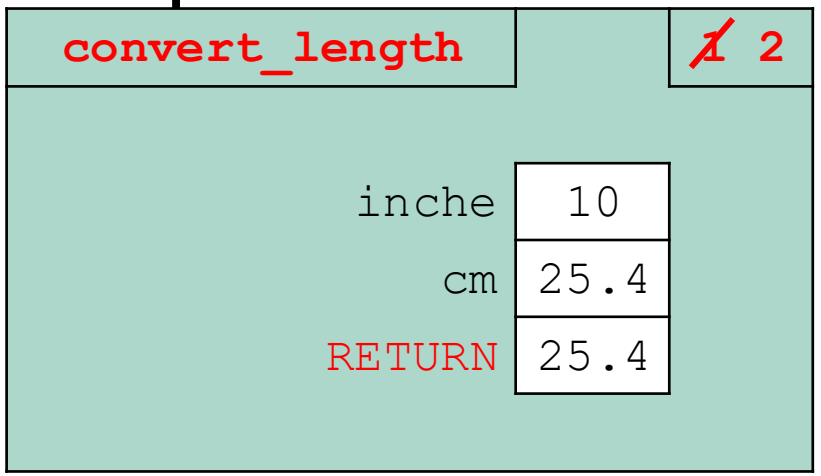
- We will draw diagrams to illustrate what is in memory
- Function frame: representation of function call



Example 1:



Example 2:



EXAMPLE 1

by drawing

PHASE 1: SET UP CALL FRAME

```
def areaTriangle(base, height):
    return 0.5 * base * height
```

```
# areaTriangle in my_poly.py module
>>> import my_poly
>>> my_poly.areaTriangle(5,10)
```

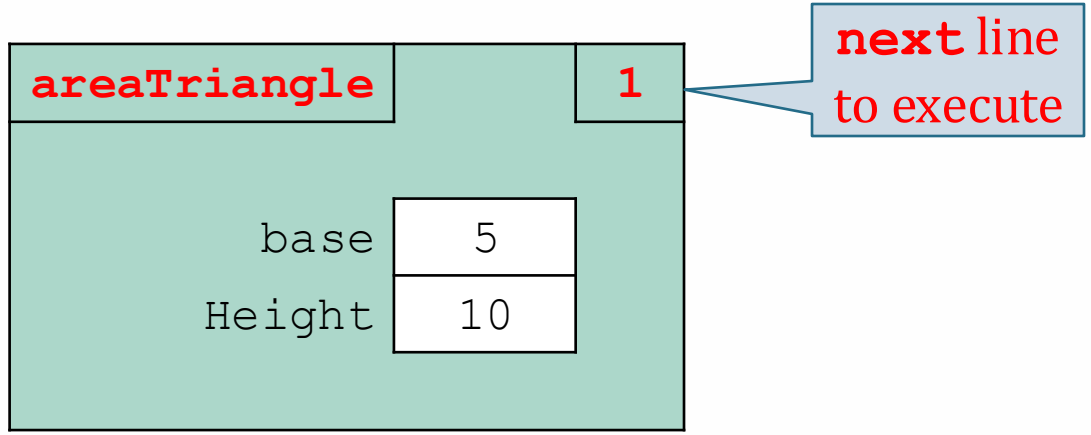
Steps:

1. Draw a frame for the argument value to the parameter (in frame)
2. Indicate next line to call
3. Assign the execute

PHASE 1: SET UP CALL FRAME

```
# areaTriangle in my_poly.py module
>>> import my_poly
>>> my_poly.areaTriangle(5,10)
```

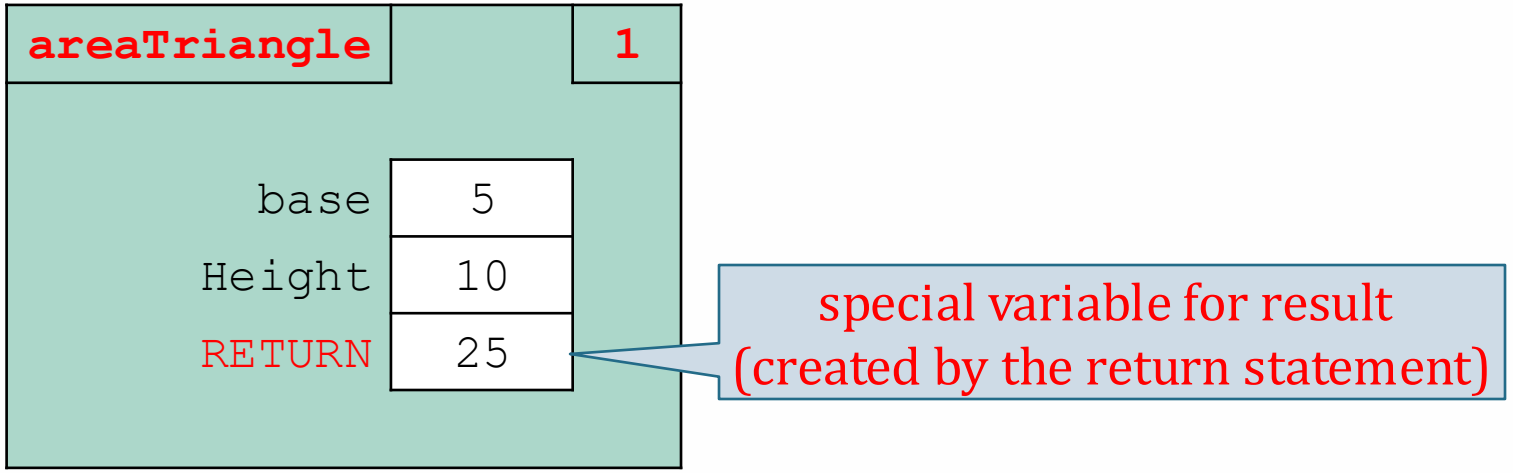
```
def areaTriangle(base, height):
1       return 0.5 * base * height
```



PHASE 2: EXECUTE FUNCTION BODY

```
# areaTriangle in my_poly.py module
>>> import my_poly
>>> my_poly.areaTriangle(5,10)
```

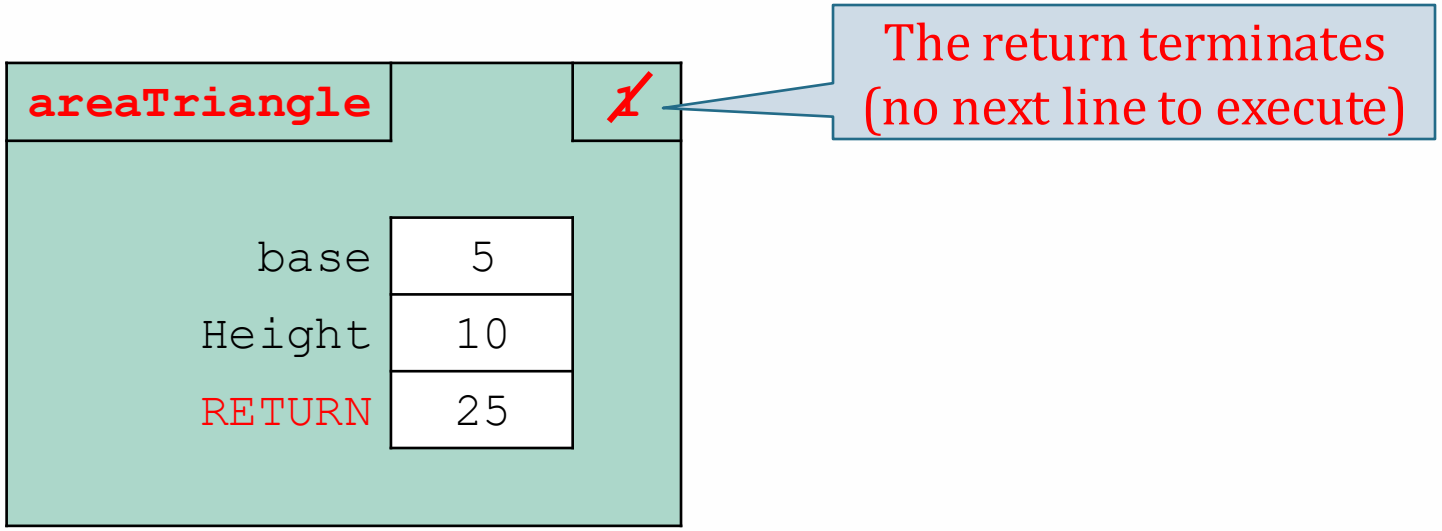
```
   def areaTriangle(base, height):
1  ➡  return 0.5 * base * height
```



PHASE 2: EXECUTE FUNCTION BODY

```
# areaTriangle in my_poly.py module
>>> import my_poly
>>> my_poly.areaTriangle(5,10)
```

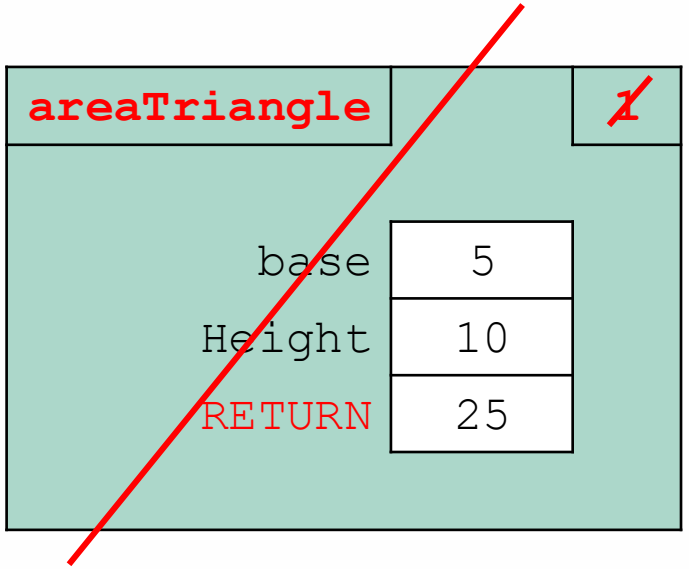
```
   def areaTriangle(base, height):
1      return 0.5 * base * height
```



PHASE 2: EXECUTE FUNCTION BODY

```
# areaTriangle in my_poly.py module
>>> import my_poly
>>> my_poly.areaTriangle(5,10)
>>> 25
```

```
   def areaTriangle(base, height):
1      return 0.5 * base * height
```

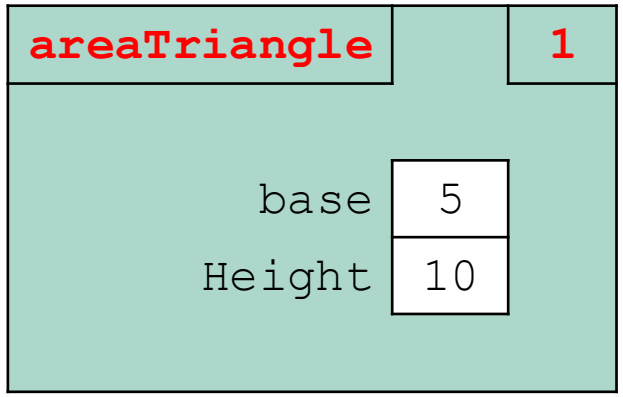


EXAMPLE 2
(WITH LOCAL VARIABLES)

by drawing

PHASE 1: SET UP CALL FRAME

```
# areaTriangle in my_poly.py module
>>> import my_poly
>>> my_poly.areaTriangle(5,10)
```



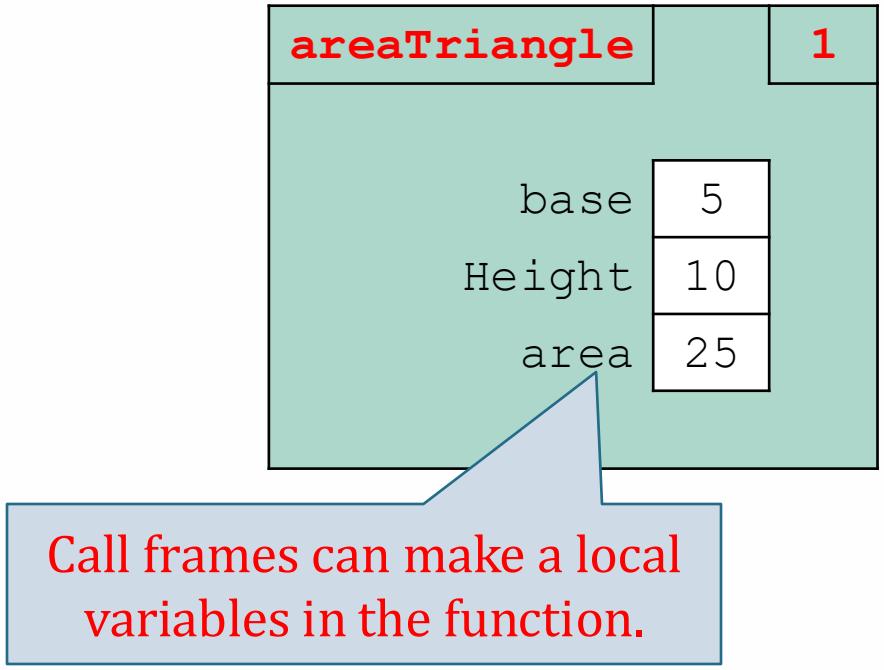

```
def areaTriangle(base, height):
    area = 0.5 * base * height
    return area
```

1

2

PHASE 2: EXECUTE FUNCTION BODY

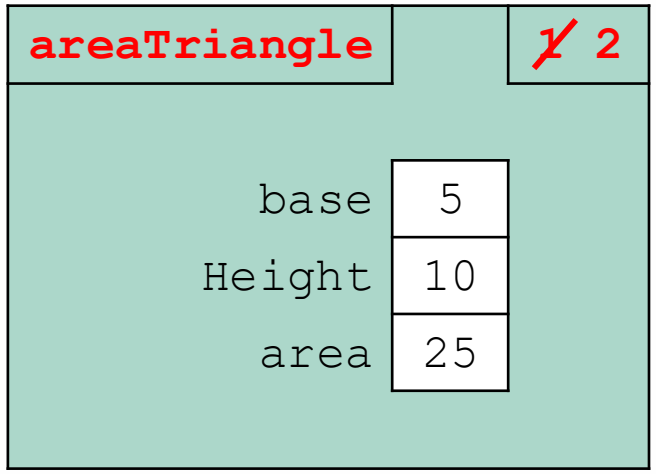
```
# areaTriangle in my_poly.py module
>>> import my_poly
>>> my_poly.areaTriangle(5,10)
```




```
def areaTriangle(base, height):
1   area = 0.5 * base * height
2   return area
```

PHASE 2: EXECUTE FUNCTION BODY

```
# areaTriangle in my_poly.py module
>>> import my_poly
>>> my_poly.areaTriangle(5,10)
```

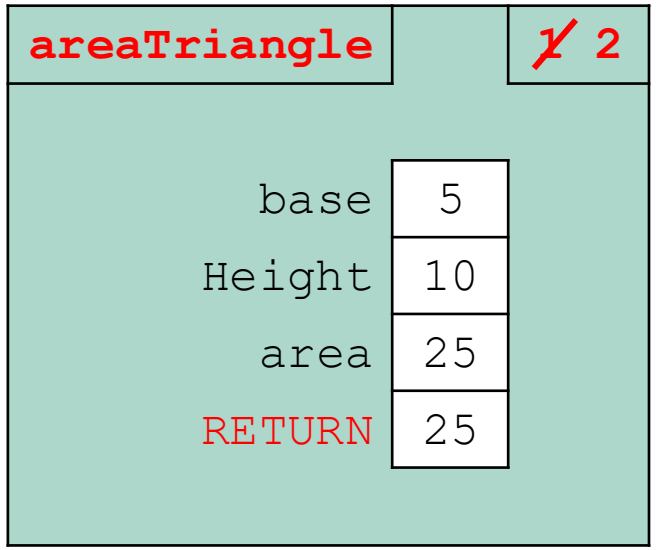



```
def areaTriangle(base, height):
    area = 0.5 * base * height
    return area
```

1 → 2

PHASE 2: EXECUTE FUNCTION BODY

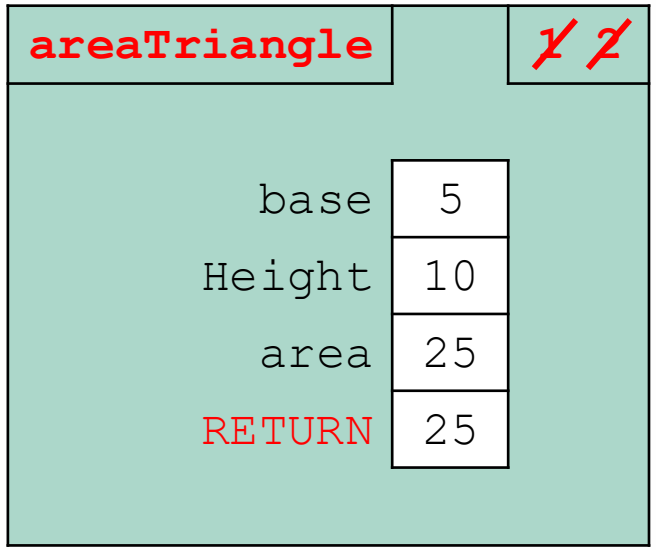
```
# areaTriangle in my_poly.py module
>>> import my_poly
>>> my_poly.areaTriangle(5,10)
```




```
def areaTriangle(base, height):
    area = 0.5 * base * height
    return area
```

PHASE 2: EXECUTE FUNCTION BODY

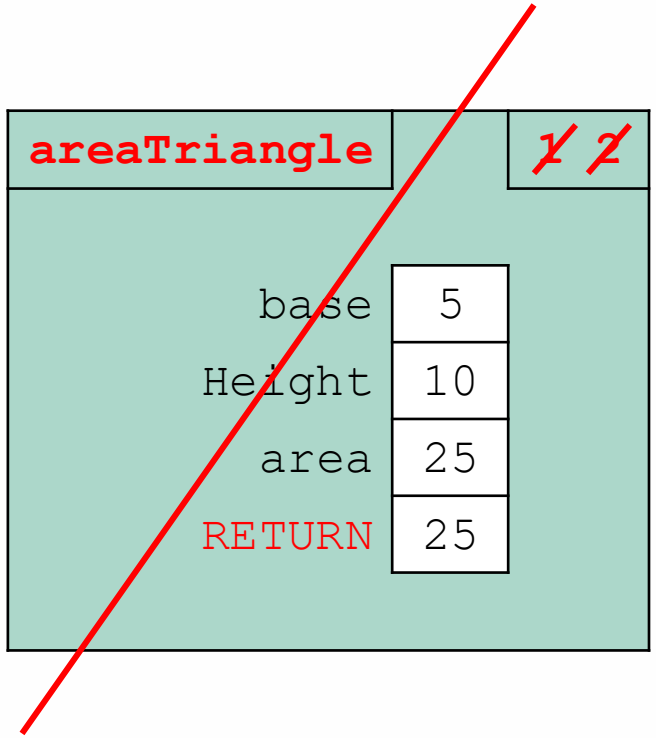
```
# areaTriangle in my_poly.py module
>>> import my_poly
>>> my_poly.areaTriangle(5,10)
```




```
def areaTriangle(base, height):
    area = 0.5 * base * height
    return area
```

PHASE 3: DELETE CALL FRAME

```
# areaTriangle in my_poly.py module
>>> import my_poly
>>> my_poly.areaTriangle(5,10)
>>> 25
```




```
def areaTriangle(base, height):
    area = 0.5 * base * height
    return area
```

1

2

PYTHON TUTOR

Python Tutor is a live programming environment that lets you visualize code execution step by step

VISUALIZE EXECUTION – PYTHON TUTOR

Python 3.6

```
1  #areaTriangle.py
2
3  """
4  script that compute the area of a triangle
5  """
6
7  def areaTriangle(base, height):
8
9      """
10     Input: base and area are positive integer
11     Return the area of a triangle using formula area
12     area = 0.5 x base x height
13     """
14     print("The area of your triangle is:")
15     return 0.5 * base * height
16
17 areaTriangle(5, 10)
```

Edit this code

line that just executed
next line to execute [source]

Print output (drag lower right corner to resize)

Frames Objects

VISUALIZE EXECUTION – PYTHON TUTOR

Python 3.6

```
1  #areaTriangle.py
2
3  """
4  script that compute the area of a triangle
5  """
6
7  def areaTriangle(base, height):
8
9      """
10     Input: base and area are positive integer
11     Return the area of a triangle using formula area
12     area = 0.5 x base x height
13     """
14     print("The area of your triangle is:")
15     return 0.5 * base * height
16
17 areaTriangle(5, 10)
```

Edit this code

line that just executed

next line to execute

Print output (drag lower right corner to resize)

Frames Objects

VISUALIZE EXECUTION – PYTHON TUTOR

Python 3.6

```
1  #areaTriangle.py
2
3  """
4  script that compute the area of a triangle
5  """
6
7  def areaTriangle(base, height):
8
9      """
10     Input: base and area are positive integer
11     Return the area of a triangle using formula area
12     area = 0.5 x base x height
13     """
14     print("The area of your triangle is:")
15     return 0.5 * base * height
16
17 areaTriangle(5, 10)
```

Edit this code

line that just executed
next line to execute

Print output (drag lower right corner to resize)

Frames Objects

Global frame
areaTriangle

function
areaTriangle(base, height)

VISUALIZE EXECUTION – PYTHON TUTOR

Python 3.6

```
1  #areaTriangle.py
2
3  """
4  script that compute the area of a triangle
5  """
6
7  def areaTriangle(base, height):
8
9      """
10     Input: base and area are positive integer
11     Return the area of a triangle using formula area
12     area = 0.5 x base x height
13     """
14     print("The area of your triangle is:")
15     return 0.5 * base * height
16
17 areaTriangle(5, 10)
```

Edit this code

line that just executed
next line to execute

Print output (drag lower right corner to resize)

Frames Objects

Global frame
areaTriangle

function
areaTriangle(base, height)

areaTriangle
base 5
height 10

VISUALIZE EXECUTION – PYTHON TUTOR

Python 3.6

```
1  #areaTriangle.py
2
3  """
4  script that compute the area of a triangle
5  """
6
7  def areaTriangle(base, height):
8
9      """
10     Input: base and area are positive integer
11     Return the area of a triangle using formula area
12     area = 0.5 x base x height
13     """
14     print("The area of your triangle is:")
15     return 0.5 * base * height
16
17 areaTriangle(5, 10)
```

Edit this code

line that just executed
next line to execute

Print output (drag lower right corner to resize)

Frames Objects

Global frame
areaTriangle

function
areaTriangle(base, height)

areaTriangle
base 5
height 10

VISUALIZE EXECUTION – PYTHON TUTOR

Python 3.6

```
1  #areaTriangle.py
2
3  """
4  script that compute the area of a triangle
5  """
6
7  def areaTriangle(base, height):
8
9      """
10     Input: base and area are positive integer
11     Return the area of a triangle using formula area
12     area = 0.5 x base x height
13     """
14     print("The area of your triangle is:")
15     return 0.5 * base * height
16
17 areaTriangle(5, 10)
```

Edit this code

line that just executed
next line to execute

Print output (drag lower right corner to resize)

```
The area of your triangle is:
```

Frames Objects

Global frame
areaTriangle → function areaTriangle(base, height)

areaTriangle
base 5
height 10

VISUALIZE EXECUTION – PYTHON TUTOR

Python 3.6

```
1  #areaTriangle.py
2
3  """
4  script that compute the area of a triangle
5  """
6
7  def areaTriangle(base, height):
8
9      """
10     Input: base and area are positive integer
11     Return the area of a triangle using formula area
12     area = 0.5 x base x height
13     """
14     print("The area of your triangle is:")
15     return 0.5 * base * height
16
17 areaTriangle(5, 10)
```

Edit this code

line that just executed
next line to execute

Print output (drag lower right corner to resize)

```
The area of your triangle is:
```

Frames Objects

Global frame

areaTriangle

function
areaTriangle(base, height)

areaTriangle

base	5
height	10
Return value	25.0

VISUALIZE EXECUTION – PYTHON TUTOR

Python 3.6

```
1  #areaTriangle.py
2
3  """
4  script that compute the area of a triangle
5  """
6
7  def areaTriangle(base, height):
8
9      """
10     Input: base and area are positive integer
11     Return the area of a triangle using formula area
12     area = 0.5 x base x height
13     """
14     print("The area of your triangle is:")
15     return 0.5 * base * height
16
17 areaTriangle(5, 10)
```

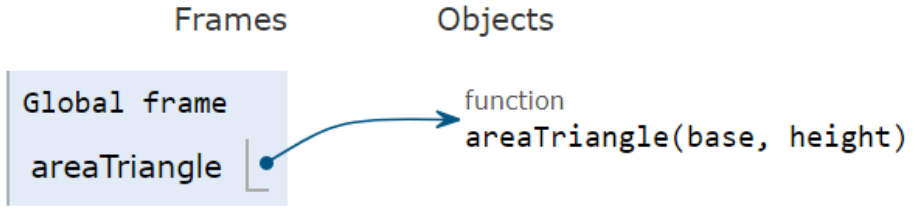
Edit this code

line that just executed

next line to execute

Print output (drag lower right corner to resize)

```
The area of your triangle is:
```



IDENTIFY AND CORRECT THE ERRORS (IF ANY)

```
def calculate_sum(a, b):
    result = a + b
    return result

print(calculate_sum(3, 5))
```

```
def greet(name):
    return "Hello, " + name

print(greet("Alice"))
print(greet("Bob"))
```

```
def find_max(numbers):
    max_num = max(numbers)
    return max_num

numbers = [4, 9, 2, 7, 5]
print(find_max(numbers))
```

THANK YOU FOR YOUR ATTENTION!

